simplifynext

ignite
simplifynext

AGENTIC AI HACKATHON 2026

Fueling Tomorrow's Changemakers

NUS National University of Singapore | Centre for Future-ready Graduates

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SMU SINGAPORE MANAGEMENT UNIVERSITY

SUTD SINGAPORE UNIVERSITY OF TECHNOLOGY AND DESIGN

Building AI Agents

Move beyond single prompts to AI agents that reason, use tools, and solve multi-step tasks.

HACKATHON WORKSHOP · 3 HOURS · HANDS-ON

Session schedule

1.5hours. Labs 01 to 02 run in the room. Labs 03 to 06 are take-home: the code is reviewed in session but not executed here.

Day 01

§1 **LLM foundations**
How LLM works, Request and Response Structure

§2 **Agentic AI**
The loop, tools, planning, memory

Prompt Engineering
Zero Shot, Few Shot, Chain of Thoughts, RAG

§3 **Bedrock**
InvokeModel, tool calling, the ReAct loop

Day 02

§4 **LangGraph**
State, nodes, edges, routing

§5 **DeepAgents and the Claude Agent SDK**
Deep Agents Sub-agents,

§6 **Agent Deployment using Bedrock AgentCore**
Configure, deploy, invoke, tear down

There will be Hands-on lab after every session.

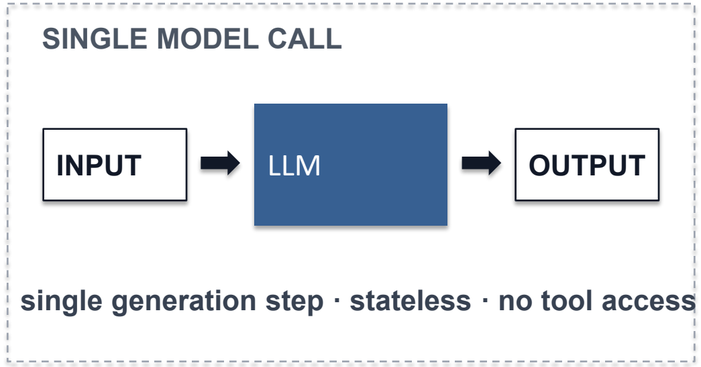
Large Language Models

Every agent is a loop around one model call. We begin with the call.

Generative AI (LLM) vs AI Agent vs Agentic AI

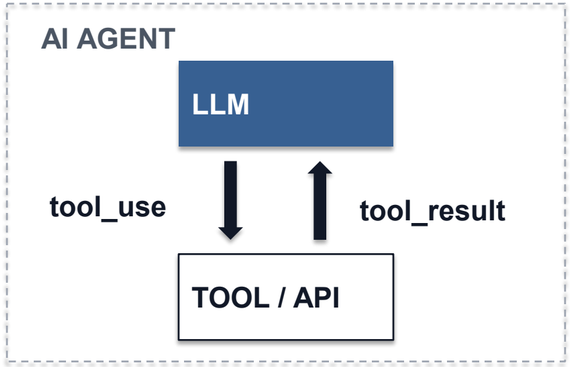
Three paradigms built on the same core engine. What separates them is tool access, orchestration, and level of autonomy, not model capability.

01 · GENERATIVE AI (LLM)



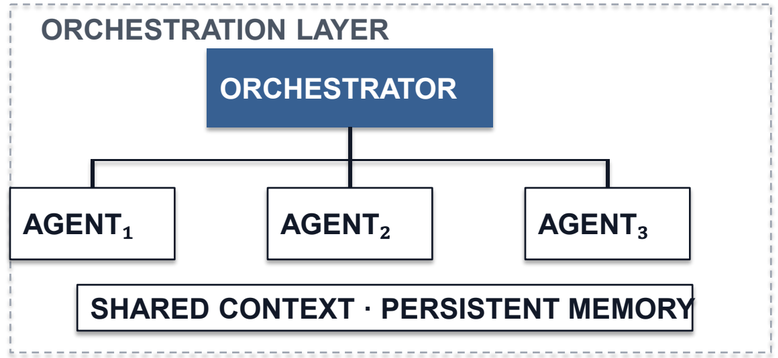
AUTONOMY	Low, prompt-driven
FLOW	Input → Output
DECISION	Pattern selection
STATE	None between calls
FAILURE	Hallucination

02 · AI AGENT



AUTONOMY	Medium, selects and invokes tools
FLOW	Input → Tool → Output
DECISION	Tool selection
STATE	Episodic, within one run
FAILURE	Tool misuse, shallow reasoning

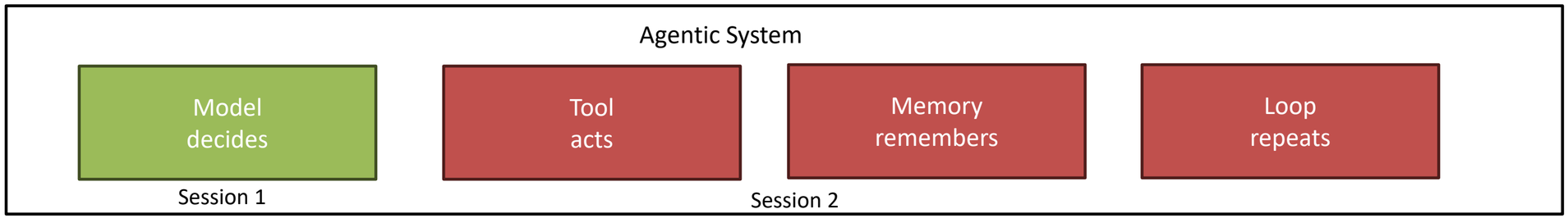
03 · AGENTIC AI



AUTONOMY	High, manages the workflow
FLOW	Input → Agent$_1$ → Agent$_2$ → Output
DECISION	Goal decomposition and assignment
STATE	Persistent, shared across agents
FAILURE	Coordination failure, drift

Taxonomy after Sapkota, Roumeliotis & Karkee, "AI Agents vs. Agentic AI", arXiv:2505.10468 (2025), Tables VI–IX.

How Large Language Models works



- It predicts the one token at a time by predicting the next token
  - Text is produced one token at a time, each conditioned on everything before it. No plan is formed in advance, and no stored answer is retrieved.
- It retains nothing → messages
  - Every call is independent. Continuity exists only because the application resends the conversation each time.
- It samples → temperature
  - Each token is drawn from a distribution. The same input may therefore yield a different answer on a second attempt.
- Its context is finite shared → usage
  - The system prompt, conversation history, retrieved documents, and tool outputs all share the same finite context window. Input and output tokens are counted and billed separately.

The Structure of a Request

A request consists of four fields. The model receives nothing beyond this object, and the server retains nothing once the response has been returned.

PART	
model	Which model serves the request. The identifier is provider-specific.
messages	The conversation, resubmitted in full each call. An agent's entire memory. Roles must alternate. The full list is resent on every call.
max_completion_tokens	A ceiling on the reply alone. Generation halts mid-structure.
temperature	Token selection. Zero requests the most probable token. (temperature=0 is not fully deterministic.)

WHAT YOU SEND

```
{
 "anthropic_version": "bedrock-2023-05-31",
 "system":     "You are a careful extractor.",
 "messages":   [ {"role": "user", "content": "..."} ],
 "max_tokens":  1024,
 "temperature": 0
}
```

WHAT COMES BACK

```
{
 "content":    [ {"type": "text",
                  "text": "..."} ],
 "stop_reason": "end_turn",
 "usage": {"input_tokens": 412,
           "output_tokens": 87}
}
```

LAB: SECTION_1_FOUNDATIONS/01_foundations.py · Part A the request body · Part C the token ceiling · Part D sampling

The Response Object

Three values return from every call. Most applications read the first and discard the other two.

PART	
content	The generated text. A string in every case, whatever structure was requested.
finish_reason	stop means the model concluded. length means the ceiling intervened and the text is incomplete.
usage	Tokens counted in each direction. The only cost figure that reflects the actual workload.

The token ceiling stops the model mid-sentence, and the text gives no sign of it. Cut prose still reads as complete. Cut JSON fails to parse, which looks like a model error but is not. *finish_reason* is the only place the difference shows.

DEFINE, THEN VALIDATE

```
def ask(client, messages, max_tokens=None,
temperature=None):
    resp = client.chat.completions.create(
            model=GROQ_MODEL,
            messages=messages,
                max_completion_tokens=MAX_TOKENS if
max_tokens is None else max_tokens,
            temperature=TEMPERATURE if temperature is None
else temperature)
    choice = resp.choices[0]
    return choice.message.content, choice.finish_reason,
resp.usage
```

Using Free Model with LangChain

Every chat model in LangChain answers to the same methods. Changing supplier is therefore a change of class and identifier and leaves the surrounding code untouched.

Provider	LANGCHAIN CLASS	NOTES
Groq	ChatGroq	Used throughout this workshop
Google AI Studio	ChatGoogleGenerativeAI	Gemini 2.5 Flash. Multimodal, Accepts image input
Cerebras	ChatCerebras	Largest daily allowance listed
OpenRouter	ChatOpenRouter	Many free models behind one key. Shared slots can queue.
Mistral	ChatMistralAI	Mistral Small. Limits are set per workspace.

```
# uv add langchain-groq
# key: console.groq.com

from langchain_groq import ChatGroq

model = ChatGroq(
    model="openai/gpt-oss-20b",
    temperature=0,
    api_key=GROQ_API_KEY,
)
```

All three Session 1 labs run on Groq. Bedrock is not used until Session 2.

LAB 01

Foundation

Calling an LLM via API Key

DURATION	FILE	REQUIRES
15 minutes	Lab/section_1_foundation/	Groq key only

BEFORE YOU ARRIVE

Prerequisites

Four steps. No API key is needed yet; that is set up during the session.

00 Follow these instructions to install Git Version Control for your respective operating system: https://git-scm.com/install/

Open your Terminal / Command Prompt / Powershell and execute the following commands:

01 git clone https://github.com/thetsuwin66/agentic_ai_hackathon_2026.git && cd agentic_ai_hackathon_2026/

02 **Linux/Mac:** curl -LsSf https://astral.sh/uv/install.sh | sh
Windows: powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

03 cd lab && uv sync
Installs the dependencies. Around twenty seconds.

04 uv run 00_check_env.py
Confirms all of the above. Stages 1 to 3 should read ok; the key is reported as missing, which is expected.

If any check fails, please raise your hand.

Code repository: https://github.com/thetsuwin66/agentic_ai_hackathon_2026

Lab

Groq Setup

- **First lab requires a Groq API key and can get from the free tier by following instruction .**
- **The free tier is rate limited, not billed.**

RUN

01 **console.groq.com → create key**

Free tier. The key is shown once, so copy it now.

02 **cd agentic_ai_hackathon_2026/lab/**

03 **echo 'GROQ_API_KEY=gsk_...' >> .env**
echo 'GROQ_MODEL=openai/gpt-oss-120b' >> .env

Adds the created api key and default model to the project's environment file

04 **cd agentic_ai_hackathon_2026/lab/section_1_foundation/**

Every Session 1 file lives here.

05 **uv sync && uv run 00_check_groq.py**

Installs dependencies, then verifies the key.

EXPECTED OUTPUT

```
============================================================
SESSION 1 ENVIRONMENT CHECK
============================================================

model llama-3.3-70b-versatile
calling the raw Groq SDK...
model said: 'ready'
[groq sdk] input=40 output=2 total=42
checking langchain-groq...
langchain said: 'ready'
[ChatGroq] input=40 output=2 total=42

============================================================
OK — you are ready for all three Session 1 labs
============================================================
```

IF THE CHECK FAILS

Confirm the key begins gsk_ and that .env sits inside agentic_teaching/lab. Raise a hand rather than debugging alone.

Hands-on Lab :

- Run Lab/session_1_foundation/01_foundations.py
- No framework, just the raw request and the raw response

PART	THE LINE TO CHANGE
A · OBSERVE	No change. Read the printed request body, then content, finish_reason and usage.
B · TODO 1	`second_turn = [q1, {"role": "assistant", "content": reply}, q2]` Carry the history: the first question, the reply, then the new question.
C · TODO 2	`MAX_TOKENS = 10` Truncate deliberately. The JSON stops mid-structure and json.loads raises.
D · TODO 3	`TEMPERATURE = 1.0` Compare the three answers against the run at 0.0.

Agentic AI

From a single model call to an autonomous agent

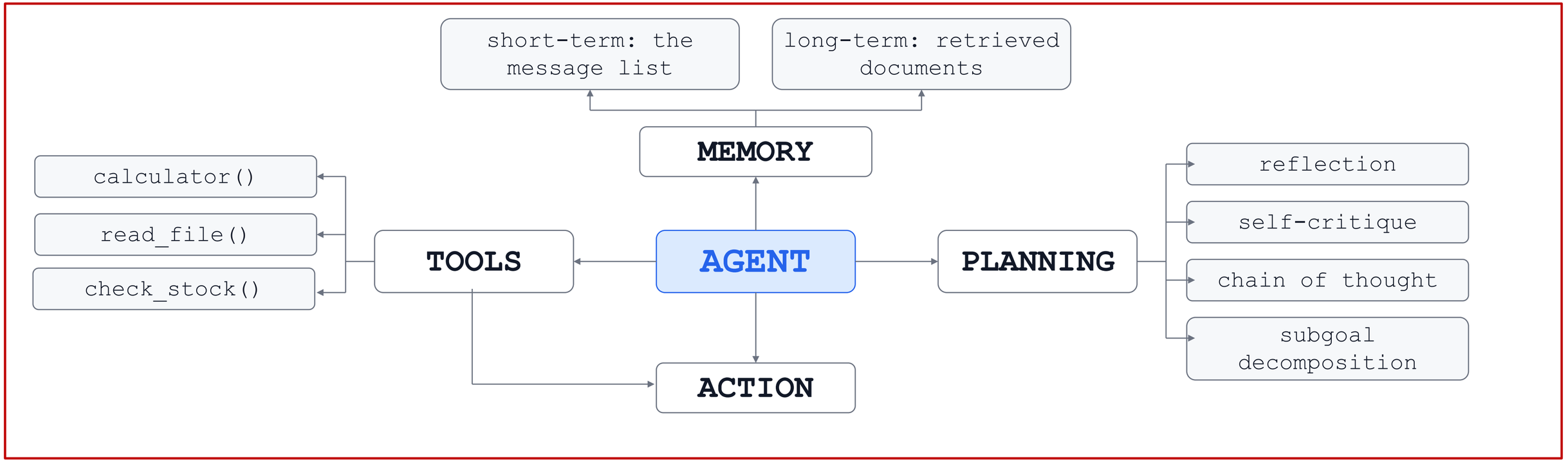
What is Agentic AI?

Agentic AI is an autonomous AI system that can act independently to achieve pre-determined goals. Traditional software follows pre-defined rules, and traditional artificial intelligence also requires prompting and step-by-step guidance. However, agentic AI is proactive and can perform complex tasks without constant human oversight. "Agentic" indicates agency — the ability of these systems to act independently, but in a goal-driven manner.

Source: AWS — aws.amazon.com/what-is/agentic-ai.

Anatomy of an agent.

Four parts around one model call: what it can do, what it remembers, how it decides, and how it acts. Only the first is contributed by the model; the remaining four constitute the application and are the developer's responsibility.



Section_2_agentic_ai_basics/00_anatomy_of_an_agent.py

Source: https://lilianweng.github.io/posts/2023-06-23-agent/

Planning : Three Strategies

Planning determines which action is taken next. The three approaches below differ in one respect only: the point at which the order of actions is settled.

THREE STRATEGIES — DISTINGUISHED BY WHEN THE ORDER IS DECIDED

01 · DECOMPOSITION

Settled before execution

The goal is expanded into numbered subtasks, and the complete sequence exists before the first action runs.

SUITED TO
Tasks whose structure is known in advance.

02 · REACTIVE

Settled one step at a time

No sequence is produced beforehand. Each action is selected from the present state, and its observation determines the next.

SUITED TO
Uncertain conditions, where the step count is unknown.

03 · HIERARCHICAL

Order fixed at two levels

A coarse plan persists for the run, while actions beneath the current step are selected reactively and the plan revised on contradiction.

SUITED TO
Most production systems, and what the frameworks implement.

section_2_agentic_ai_basic/04_planning.py demonstrates all three

Planning : Decomposition

One model call returns the entire sequence before any tool runs. Everything after it is iteration over a list that happens to have come from a model.

ONE GOAL, EXPANDED UNTIL THE LEAVES ARE ATOMIC

Goal
Decides whether to restock “SKU 777”

01 Check the stock level

02 Obtain the supplier lead time

03 Conclude restock or hold, with the reason

The plan comes back as a typed object, not a string to parse. Prompting for JSON works until the model wraps its reply in a code fence

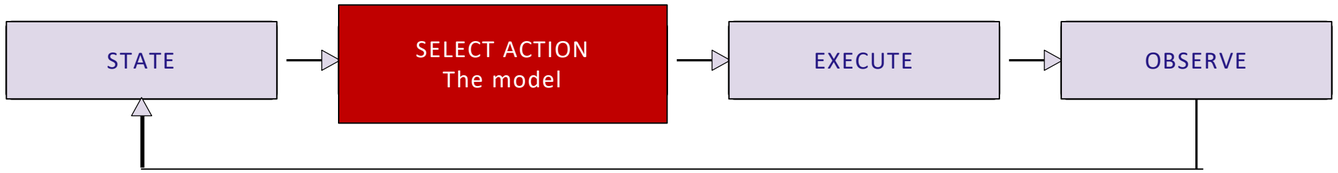
```
class Plan(BaseModel):
    steps: list[str]                    # strings, not tool calls

planner = chat_model(temperature=0).with_structured_output(Plan)   # no tools
actor   = chat_model(temperature=0).bind_tools(TOOLS)              # tools

# PLAN: one call, before anything runs
steps = planner.invoke([
    SystemMessage(PLANNER),
    HumanMessage(GOAL),
]).steps

# EXECUTE: a plain for-loop; enumerate decides the order, not a model
messages = [SystemMessage(SYSTEM_STEPWISE)]

for i, step in enumerate(steps, 1):
    messages.append(HumanMessage(f"Step {i}: {step}"))
    messages = react(messages, actor, max_steps=3)
```

Section_2_agentic_ai_basics/04a_planning_decomposition.py · lines 63–78, 123–134

Planning : Reactive

No sequence is produced in advance. One action is selected from the state as it stands, and the observation it returns becomes the state the next selection is made from.

ONE ITERATION, REPEATED



THE OBSERVATION IS THE NEXT STATE

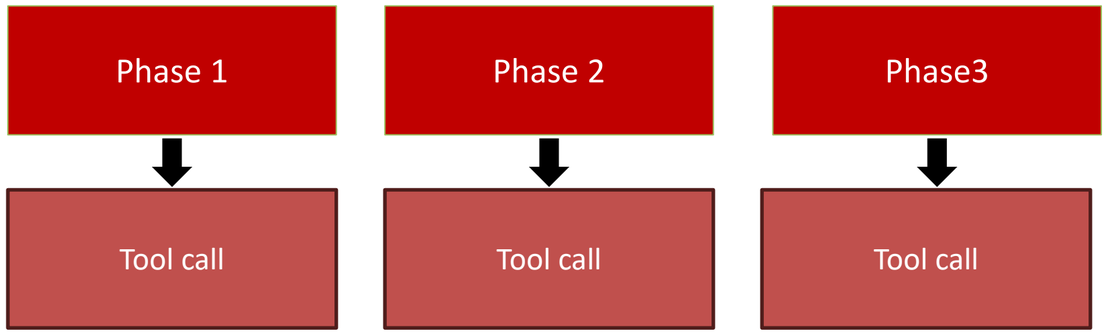
```
model = chat_model(temperature=0).bind_tools(TOOLS)      # one model, sees GOAL

messages = [SystemMessage(SYSTEM), HumanMessage(GOAL)]   # goal stays in context

for step in range(1, MAX_STEPS + 1):                      # a cap, not a plan
    reply = model.invoke(messages)                        # decide from transcript
    messages.append(reply)

    if not reply.tool_calls:                              # no tool = done
        break

    for call in reply.tool_calls:
        result = BY_NAME[call["name"]].invoke(call["args"])
        messages.append(ToolMessage(result, tool_call_id=call["id"]))
```

- **There is no planning call.** The loop is the strategy. Selection happens inside the model call, and the code contributes only execution and the append that follows.
- **The step count is unknown until the run ends.** The range is the only bound, and without it a run that fails to converge does not stop.

Section_2_agentic_ai_basics/ 04b_planning_reactive.py · lines 65–79

Planning : Hierarchical

The model produces two or three broad phases at the start, and that plan is kept for the whole run. Within each phase, the agent works reactively, choosing one action at a time. The plan changes only when a result makes the remaining phases wrong.




```
plan = plan_steps("Give 2-3 coarse phases. Phases, not tool calls.", GOAL)

i = 0
while i < len(plan):                    # not a for-loop: plan can change
    messages.append(HumanMessage(f"Work on this phase only: {plan[i]}"))
    messages = react(messages, actor, max_steps=3)

    if contradicts(messages[-1], plan[i + 1:]):
        plan = plan[:i + 1] + plan_steps("Revise the REMAINING phases.", ...)
    i += 1
```

- **Two loops, two bounds.** The outer loop walks the phases. The inner loop gets three steps to finish each one, and that allowance starts again at every phase..
- **The revision check is plain Python.** It looks for a word like "cannot" in the reply. No second model call, so it costs nothing and answers the same way every time.
- **Only the unfinished phases are rewritten.** Completed phases stay as they are. The rewrite covers the rest, and the model is told the goal, what is done, and what went wrong.

04_planning.py · lines 167–206 · the arrangement the frameworks in §4 and §5 implement

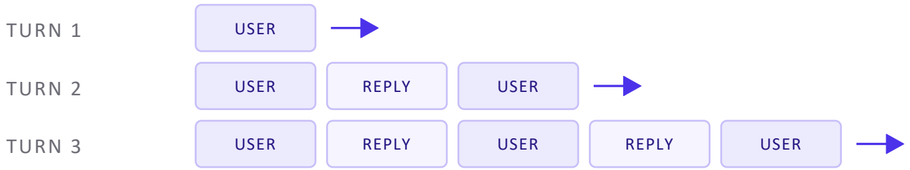
Memory : Maintaining conversation context

The model retains nothing between calls. Short-term memory is the messages list, sent in full with every request. Long-term memory is storage outside the model, queried and inserted into that same list.

TWO KINDS OF MEMORY

SHORT TERM · THE CONTEXT WINDOW

The messages list itself



EACH REQUEST CARRIES EVERY EARLIER TURN

Between 4,000 and 200,000 tokens depending on the model. It holds the conversation, recent observations and intermediate results.

```
reply = ask(messages)            # turn 1
messages.append(assistant(reply))
messages.append(user(next_question))
reply = ask(messages)  # the whole list again
```

LONG TERM · OUTSIDE THE MODEL

A store queried each turn

RANKED BY SIMILARITY, NOT BY RECENCY

Entries are held as embeddings, so a new problem retrieves comparable earlier ones. Content reaches the model only once retrieval places it in the list.

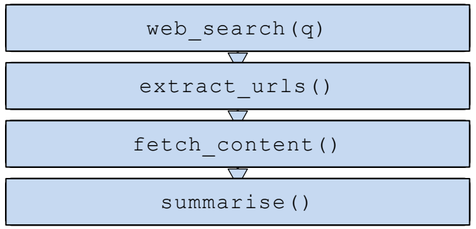
```
hits = store.search(embed(q), k=3)  # by meaning
messages.insert(1, system(join(hits)))
reply = ask(messages)
store.add(embed(summary(reply)), reply)
```

Long-term memory is a retrieval problem before it is a storage one: what the model can use on a given turn is whatever the search returned, and entries that are never retrieved have no effect on the reply.

Tool : Composition

One call is rarely enough. When several tools are available, the model combines them in three shapes. These shapes are not written by the developer. The model produces them, and the loop runs whatever it requests.

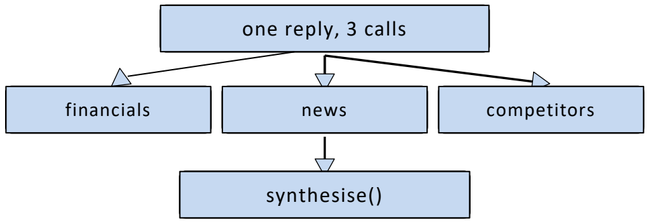
-> 01 · SEQUENTIAL



LATENCY IS THE SUM OF THE STEPS

Each result becomes the next input. Check the stock, then look up the lead time for whatever it returned.

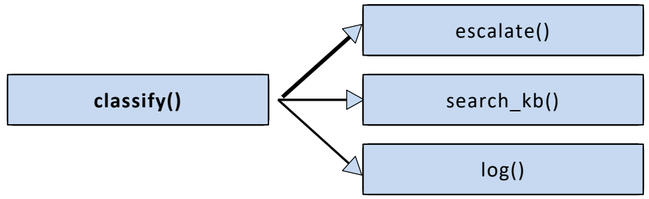
|| 02 · PARALLEL



LATENCY IS THE SLOWEST CALL

One reply asks for several independent calls at once. Nothing depends on anything else.

?: 03 · CONDITIONAL



ONE BRANCH EXECUTES · THE OTHERS NEVER RUN

One result decides which tool comes next. Classify first, then escalate, search, or log.

Tool : Reaching outside the model

A tool is an ordinary function paired with a schema. The model reads the schema only. The function runs in the application, and a reply carries a request to invoke it rather than a result.

Section_02_agentic_ai_basics/02_tools.py

{ } DECLARATION · WHAT THE MODEL READS

NAME	DESCRIPTION	PARAMETERS	RETURNS

```
@tool
def get_order(order_id: str) -> str:
    """Look up one sales order by
    its id, e.g. SO-1001. Returns
    status and total."""
```

function name	→	name
docstring	→	description
Type hint	→	parameter

( ) INVOCATION · WHAT ACTUALLY RUNS

```
for call in reply.tool_calls:
    result = BY_NAME[call["name"]] \
        .invoke(call["args"])
    messages.append(ToolMessage(
        result,
        tool_call_id=call["id"]))
```

- The model returns a name and a JSON object.
- Your code decides whether to run it.
- The result is appended, then sent back.

The docstring is not a comment. It is the prompt. It is what the model reads when deciding which tool to pick, so a vague docstring is a leading cause of an agent calling the wrong thing.

Action : execution, observation, repeat

- Action is the only step that changes the outside world, and the only step that can fail. Two failures are common: the model names a tool that does not exist or supplies arguments that make one raise.

```
for call in reply.tool_calls:
    fn = BY_NAME.get(call["name"])

    if fn is None:
        result = f"No tool named {call['name']}. Available: {', '.join(BY_NAME)}."
    else:
        try:
            result = fn.invoke(call["args"])
        except Exception as exc:
            result = f"{call['name']} failed: {exc}"

    messages.append(ToolMessage(result, tool_call_id=call["id"]))
```

INFO · THINK, ACT, OBSERVE

Each tool result returns as a message, then the model decides again.

The loop ends when no tool is requested.

CAUTION · THE CAP IS THE SAFETY NET

A missing tool and a broken call are both written into result and appended. Nothing raises. The model reads what went wrong and can correct the call on the next step.

LAB 02

Agentic AI

Basic Components

DURATION	FILE	REQUIRES
15 minutes	Section_02_agentic_ai_basic/	Groq key only

Total Lab : 6 scripts, one component each

Each file is a short script you can run on its own.

```
section_2_agentic_ai_basic/
├── _bootstrap.py
├── README.md
├── 00_anatomy_of_an_agent.py
├── 01_memory.py
├── 02_tools.py
├── 03_tool_composition.py
├── 04_planning.py
├── 05_agent_lab.py
└── 05_agent_lab_solution.py
```

RUN IN ORDER 00 → 05

FILE	WHAT THE SCRIPT DOES
00_anatomy_of_an_agent.py	Runs a real model in a loop, calling tools until it has an answer.
01 memory.py	Contrasts stateless, Short-term memory, and retrieval-based context for a single query.
02 tools.py	Shows what a tool looks like to the model, and what asking for one looks like coming back.
03 tool_composition.py	Runs three tools chained, in parallel, and behind a branch, timing the wall clock and round trips for each.
04 planning.py	Answers one restock question three ways — plan first, step by step, both — then prints messages, tokens and seconds.
05 agent_lab.py	The hands-on file. Ships working and useless — nothing enters the transcript, so the model asks the same question six times. Four TODOs make it an agent.

Prompt Engineering

Eliciting higher quality LLM responses

Zero Shot Prompting

- Describe the task and ask. No examples are supplied. This is the baseline, and it is correct far more often than people expect.

```
model = chat_model(temperature=0)

reply = model.invoke([
   SystemMessage(
                  "Classify the ticket as billing, technical or account. "
                  "Reply with one word."),
   HumanMessage("My card was charged twice this month."),
])
# -> 'billing'
```

USE THIS WHEN

The task can be stated in a sentence, and the categories mean what their names suggest.

Pros and Cons

- **Pros:** Fast, simple, and saves token space because your prompt is short.
- **Cons:** Can fail or be less accurate on complex, niche, or very specific formatting tasks

When zero-shot doesn't work, it's recommended to provide demonstrations or examples in the prompt which leads to few-shot prompting. In the next section, we demonstrate few-shot prompting.

Lab/section_2_agentic_ai_basic/06a_zero_shot.py

Source : https://www.promptingguide.ai/techniques

Few Shot Prompting

- Show two or three worked examples before the real input. Examples teach a convention that a description struggles to convey: which category a borderline case belongs to, and exactly how the answer should be written.

```
SYSTEM = """Classify the ticket as billing, technical or account.
Reply with one word.

The app crashes on login   -> technical
I was charged twice        -> billing
I cannot reset my password -> account"""

reply = model.invoke([SystemMessage(SYSTEM),
                      HumanMessage("Login page returns a 500 error.")])
# -> 'technical'
```

USE THIS WHEN

A borderline case keeps going the wrong way. Notice that "cannot reset my password" is filed under account, not technical, and no description would have settled that.

Pros and Cons

- **Pros:** Settles borderline cases that words cannot. Fixes the output format by showing it. No training or setup needed.
- **Cons:** Costs input tokens on every call. In an agent the system prompt is resent at each step, so you pay repeatedly.

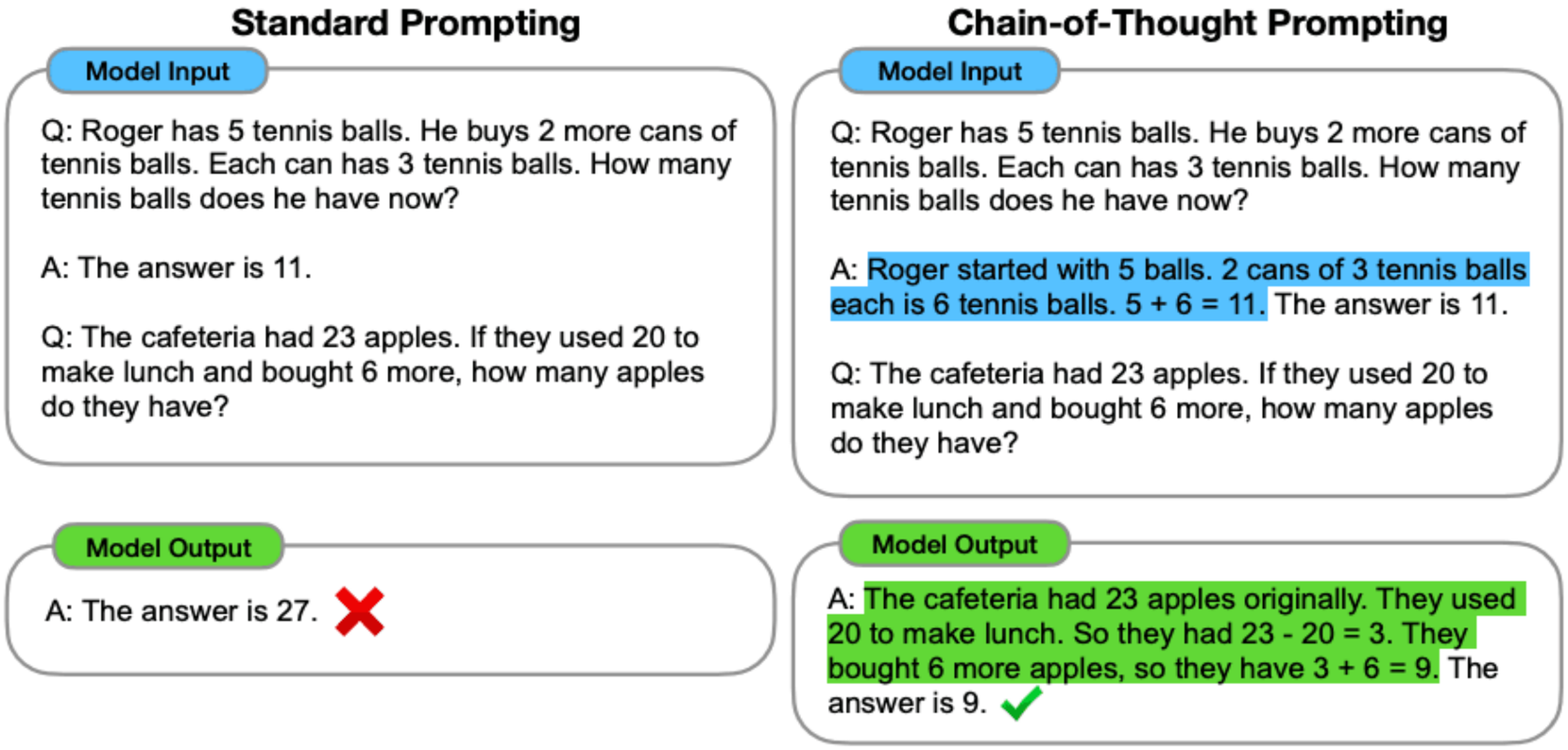
One careless example is worse than none. The model follows the examples over the written rule. Notice that "cannot reset my password" is filed under account, not technical; no description would have settled that, but one example does.

Lab/section_2_agentic_ai_basic/06b_few_shot.py

Source : https://www.promptingguide.ai/techniques

Chain of Thought (CoT) Prompting

- Introduced in Wei et al. (2022), chain-of-thought (CoT) prompting enables complex reasoning capabilities through intermediate reasoning steps. You can combine it with few-shot prompting to get better results on more complex tasks that require reasoning before responding.



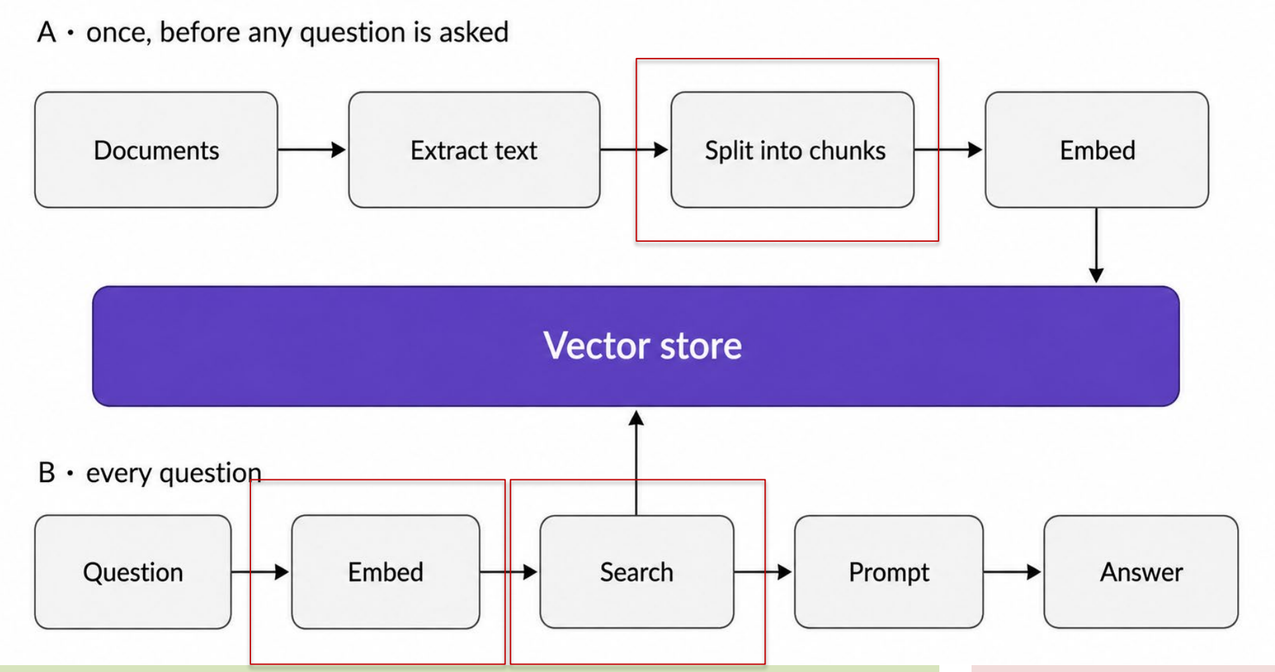
Current models mostly do this already. Wei et al. described it as an ability that appears only in large models; today's models reason unprompted, so the working now arrives whether you ask or not.

Lab/section_2_agentic_ai_basic/06c_cot.py

Source : https://www.promptingguide.ai/techniques

Retrieval Augmented Generation (RAG)

- **Relevant text is retrieved first and placed in the prompt, so the answer is drawn from supplied material rather than from training data. The work divides into two phases that run at different times.**



	In the lab
Chunking	Two sentences per chunk, split on punctuation.
Embedding	A normalised word count. Matches shared words only.
Ranking	Cosine similarity, take the top three above a score floor.

The lab takes the simplest option at each step. All three have many alternatives. These are worth reading about if your project leans on retrieval.

PROS

Answers reflect current and private material. Sources can be cited. Documents are updated by re-running phase A, with no retraining.

CONS

Phase A is a system to build and maintain. Retrieve the wrong passages and the answer is confidently wrong. Every passage is billed as input.

Lab/section_2_agentic_ai_basic/06d_rag.py

Source : https://www.promptingguide.ai/techniques

There are many more ..

- Four were covered here. The rest are catalogued below, and several are worth an hour of your time. ReAct in particular is the loop already built in Session 2, described as a prompting technique.

COVERED TODAY

Zero Shot | Few Shot | Chain of Thought | Retrieval Augmented Generation

THERE ARE MANY MORE

Self-Consistency | Prompt Chaining | Reflexion | Tree of Thoughts | Meta Prompting

Automatic Reasoning and Tool use | Automated Prompt Engineer | Active Prompt | Multimodal CoT | Graph Prompting

Reach for a technique only when the simple version has been shown to fail. Every one of these costs input tokens on every call, and in an agent the system prompt is resent at every step, so the cost is paid repeatedly.

Source : https://www.promptingguide.ai/techniques

Lab/section_2_agentic_ai_basic/06d_rag.py

LAB 03

Prompt Engineering

Zero Shot, Few Shot, CoT

DURATION	FILE	REQUIRES
15 minutes	Section_02_agentic_ai_basic/	Groq key only

Total Lab : 5 scripts, one exercise lab

Each file is a short script you can run on its own.

```
section_2_agentic_ai_basic/
├── _bootstrap.py
├── README.md
├── 06_prompt_engineering_lab_solution.py
├── 06_prompt_engineering_lab.py
├── 06a_zero_shot.py
├── 06b_few_shot.py
├── 06c_cot.py
├── 06d_rag.py
```

RUN IN ORDER 00 → 05

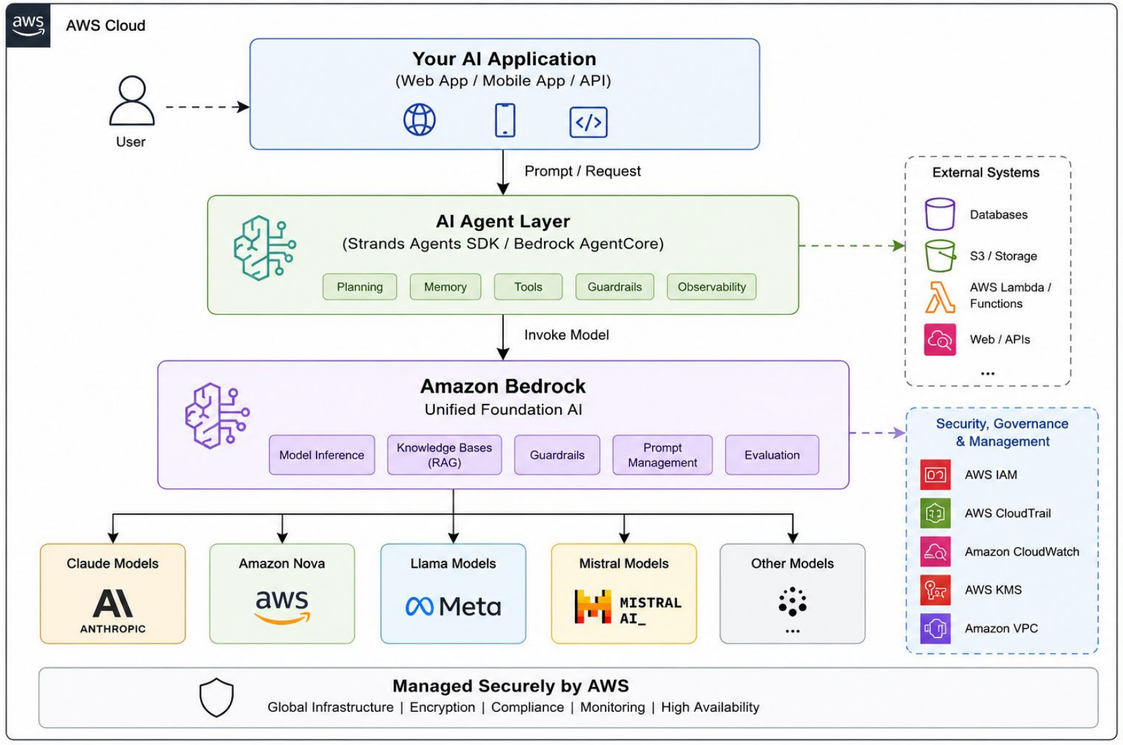
FILE	WHAT THE SCRIPT DOES
06a_zero_shot.py	Zero Shot Prompting
06b_few_shot.py	Few Shot Prompting
06c_cot.py	Chain Of Thought Prompting
06d_rag.py	Retrieval Augmented Generation
06_prompt_engineering.py	Hands-on file. Three prompts, three scores to beat. Challenge 2 locks the instruction, so examples are the only lever you have.

AWS Bedrock

Section 2 defined the agent. Section 3 build one against the Amazon Bedrock; the raw API call first, then tool use, with no framework in between

Amazon Bedrock: The Foundation for Agentic AI

One managed endpoint in front of many model vendors. No servers, no downloads, no fine-tuning. You sign an HTTPS request and pay per token.



01 · ONE RUNTIME CLIENT

A boto3 bedrock-runtime client posts signed requests to a regional endpoint. Nothing runs locally.

02 · MANY VENDORS, ONE DOOR

Anthropic, Meta, Mistral and Amazon Nova sit behind the same client. Only the modelId string changes.

03 · INVOKEMODEL VS CONVERSE

InvokeModel takes each vendor's own JSON body. Converse takes one shape for all of them.

04 · GOVERNED LIKE ANY AWS SERVICE

IAM gates calls through bedrock:InvokeModel, and each response reports input and output token usage.

CHECK ACCESS BEFORE THE FIRST CALL

Access is granted per model and per region, so valid credentials do not guarantee a working call. Verify both with section_1_foundation/00_check_bedrock.py before starting.

InvokeModel: the required keys.

- **boto3 against bedrock-runtime, with no framework in between. Every abstraction above this level reduces to the same keys, which is why it is worth reading once.**

```
body = json.dumps({
    "anthropic_version": "bedrock-2023-05-31",
    "messages": [{"role": "user",
                  "content": prompt}],
    "max_tokens": max_tokens,
    "temperature": 0,
})

response = client.invoke_model(
    modelId=model_id,
    body=body,
    contentType="application/json",
    accept="application/json",
)

envelope = json.loads(response["body"].read())
text = envelope["content"][0]["text"]
```

section_3_bedrock/01_invoke_model.py · lines 30–59

{ } FOUR KEYS ON THE CALL

modelId	which model, and which routing prefix
body	the prompt payload, serialised
contentType	what is being sent
accept	what should come back

CAUTION · TWO EASY MISTAKES

body is a JSON string, not a dict. The response body is a stream, and it can only be read once.

CONVERSE WRAPS ALL OF THIS

The same fields under different names: system, inferenceConfig, toolConfig. §2 used it through LangChain.

Choosing a model : define the metric first

- **Capability, cost and latency are the three axes. Only the model identifier changes between runs, so two models can be compared by measurement.**

01 · CAPABILITY

Whether the model holds the task at all: following instructions, formatting tool calls, recalling a long history.

02 · COST

Billed per token in each direction. Output is the expensive half, and every step resends the whole history as input.

03 · LATENCY

Time to first token and time to finish, multiplied by the number of steps the loop takes.

Which model, and what it costs.

Per million tokens. Output is five times input, on all of them. An agent loop calls the model five to fifteen times for one task, so multiply everything below.

WHAT YOU PICK	USE IT FOR	IN / OUT
global.anthropic.claude-haiku-4-5-20251001-v1:0	Classification, extraction, anything high-volume. Your default tonight.	$1 / $5
Sonnet tier	Ambiguous images, harder reasoning, when Haiku is measurably wrong	$3 / $15
Opus tier	Rarely, and probably not during a hackathon	$5 / $25
global. prefix	Cross-region routing. The regional prefix costs about 10% more	read it, never build it
Prompt caching	A repeated system prompt, billed at up to 90% less on cache reads	worth it above ~1k tokens

LAB 03

AWS SSO Setup Up

Basic Components

DURATION	FILE	REQUIRES
15 minutes	Section_02_agentic_ai_basic/	Groq key only

AWS CLI/SSO Setup

Session 1 & 2 requires none of it.

FOUR PREREQUISITES

FULL DETAIL IN THE REPO README

01 AWS CLI, version 2

Installed through Homebrew, the MSI, or the Linux zip. Version 1 does not support SSO properly and has to be removed first.

02 An SSO profile

Four values come from whoever administers the account: sso_start_url, sso_region, sso_registration_scopes> **nano ~/.aws/config**

03 An active SSO login

Sessions expire after eight to twelve hours, so day two opens with a fresh login. Nothing is written to disk. > aws sso login –profile xxxxx

04 Bedrock model access

Granted per model and per region, and off by default. Anthropic Claude Haiku 4.5, in ap-southeast-1.

ONE COMMAND NAMES THE PIECE THAT BROKE

uv run 00_check_bedrock.py

It exercises the profile, the login and model access in turn, then reports the first one that failed.

The above four cards serve as a setup checklist. Please refer to the file: agentic_ai_hackathon_2026/lab/AWS_SETUP.md for the full setup instructions.

Cost : Pricing and monitoring

Bedrock bills per token, per model, per region. Output costs several times input

FIVE PLACES, THREE MOMENTS

$ Before you choose

Bedrock pricing page[1]

Per-token rates for every model, by region.

AWS Budgets

An alert when spend crosses a threshold.

While it runs

usage on the response

Exact input and output tokens, per call.

CloudWatch metrics

Token counts per model, over time.

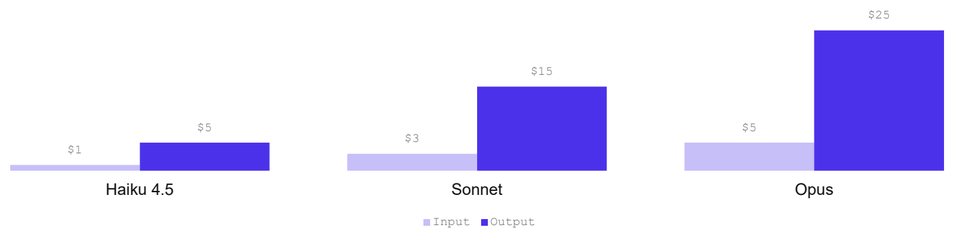
After the fact

AWS Cost Explorer

Actual spend, filtered to Amazon Bedrock.

Two days of lag is normal, so it settles late.

USD PER MILLION TOKENS · OUTPUT IS FIVE TIMES INPUT




```
usage = client.converse(...)["usage"]
usage["inputTokens"], usage["outputTokens"]   # per call
```

THE ONLY FIGURE AVAILABLE WITHOUT LEAVING THE PROCESS

Every response carries the token counts for that call. Logging them turns cost into something measured per prompt rather than discovered at the end of the month.

AWS.AMAZON.COM/BEDROCK/PRICING · DOCS.AWS.AMAZON.COM/BEDROCK/LATEST/USERGUIDE/COST-MANAGEMENT.HTML

Rates move and none of them is flat, so the arithmetic belongs on the pricing page before a run and in Cost Explorer after it

[1]https://aws.amazon.com/bedrock/pricing/

Cost and usage graph Info

Total cost	Average monthly cost	Usage type count
$7.10	$1.77	22

Report parameters

Time

Date Range

2025-01-01 — 2025-04-17

Displaying year to date

Granularity

Monthly

Group by

Dimension Clear

Usage type

Filters Info

Advanced options

Cost and usage graph Info

Total cost
$7.10

Average monthly cost
$1.77

API operation count
4

Report parameters

Time

Date Range
2025-01-01 — 2025-04-17
Displaying year to date

Granularity
Monthly

Group by

Dimension Clear
API operation

Filters Info

Advanced options

LAB 04

AWS Bedrock Lab

Basic Components

DURATION
15 minutes

FILE
Section_3_bedrock/

REQUIRES
Bedrock key required

AWS Bedrock Setup

Install, configure, log in and verify.

RUN THESE

```
# 1 · the CLI must report 2.x, not 1.x
aws --version

# 2 · profile: start URL, SSO region, account, role
aws configure sso
#     CLI default client Region  -> ap-southeast-1
#     CLI profile name           -> workshop

# 3 · log in, then confirm who the session is
aws sso login --profile workshop
aws sts get-caller-identity --profile workshop

# 4 · point the lab at Bedrock, in lab/.env
LLM_PROVIDER=bedrock
AWS_PROFILE=workshop
AWS_DEFAULT_REGION=ap-southeast-1

uv sync --extra aws
uv run 00_check_bedrock.py       # -> OK before every lab
```

CAUTION · VERSION 1 DOES NOT SUPPORT SSO

An old pip install of awscli can shadow version 2. If aws --version reports 1.x, remove it before configuring anything.

CAUTION · THE REGION IS SET TWICE

The SSO region and the CLI default client Region are separate settings. Bedrock calls follow the second one, and leaving it blank is the usual reason a call works for one person and fails for another.

MODEL ACCESS IS GRANTED IN THE CONSOLE

Bedrock, then Model access, then Claude Haiku 4.5.

Total Lab : 6 scripts, one component each

Each file is a short script you can run on its own.

```
section_3_bedrock/
├── _bootstrap.py
├── images/
├── 01_invoke_model.py
├── 02_tool_definition.py
├── 03_react_loop.py
├── 04_cost_and_optimisation.py
└── 05_multimodal.py
```

RUN IN ORDER 01 → 05

FILE	WHAT THE SCRIPT DOES
01 invoke_model	Calls a Bedrock model once and asks it what Bedrock is, then prints the reply, the stop reason and the token counts.
02 tool_definition	Gives the model a get_weather tool, asks for the weather in Singapore, and shows it requesting the call rather than answering.
03 react_loop	Runs the tool the model asked for, sends the result back, and lets it answer with a temperature.
04 cost_and_optimisation	Sends one ticket-classification prompt to Haiku and Sonnet, then trims and caches it, printing the cost of each version.
05 multimodal	Sends a photo and a question in the same message and prints the one-sentence description that comes back.

AGENTIC AI HACKATHON 2026

Thank You

Fueling Tomorrow’s Changemakers

www.simplifynext.com